



Object Oriented Programming COMP6699001

Lecturer: Jude Joseph Lamug Martinez, MCS

Final Project Report

Untitled Building Shooter

- Samuel Reagan Purnama 2902671833
 - Jose Maxwell Wei 2902739034
- Jayden Vance Pereira 3002817226

Background

For our final project, we decided to make an application that is basically 2D fortnite or more specifically a 2D battle royale game where the last man standing wins. The goal of this project is to be able to let users not just play the game, but also track statistics of every match and player. This allows them to predict the outcome of which player wins if they were to go head to head. It will display the prediction of which player wins by percentage.

Problem Description

The project addresses two interconnected problems: the design and implementation of a functioning real-time game, and the development of a companion data management tool for tracking player performance across multiple game sessions.

Game Design Problem

Designing a real-time top-down shooter requires solving several non-trivial technical challenges simultaneously:

- Tile-based collision detection — the player and enemies must move freely within the map while being blocked by walls placed on tile edges. A naive approach of checking every wall on every frame would not scale; instead, the system must efficiently determine only which walls are relevant to a given movement direction.
- Enemy AI and pathfinding — enemies must be able to navigate around walls to reach the player. This requires a graph traversal algorithm (BFS) operating over the tile grid, with results used to guide per-frame movement decisions.
- Hitscan shooting — projectiles in the game are not physical objects that move over time. Instead, they are resolved instantly by casting a ray from the shooter's position and checking for wall or character intersections along the ray's path.

Tournament Tracker Problem

Beyond the game itself, a secondary problem was identified: players who use the game for informal tournaments have no structured way to record, compare, or predict match outcomes. Specifically:

- Match results — kills, placement, and total participants — need to be recorded per player and persisted between sessions.
- Player rankings must be sortable and searchable across multiple metrics (kills, win rate, average placement, ELO score).
- A win probability predictor should be available for any two-player matchup, derived from their recorded statistics.
- All data must be saved to disk in a human-readable format and correctly restored on the next application launch.

Solution - Program Design

Goals

The solution was designed around the following primary goals:

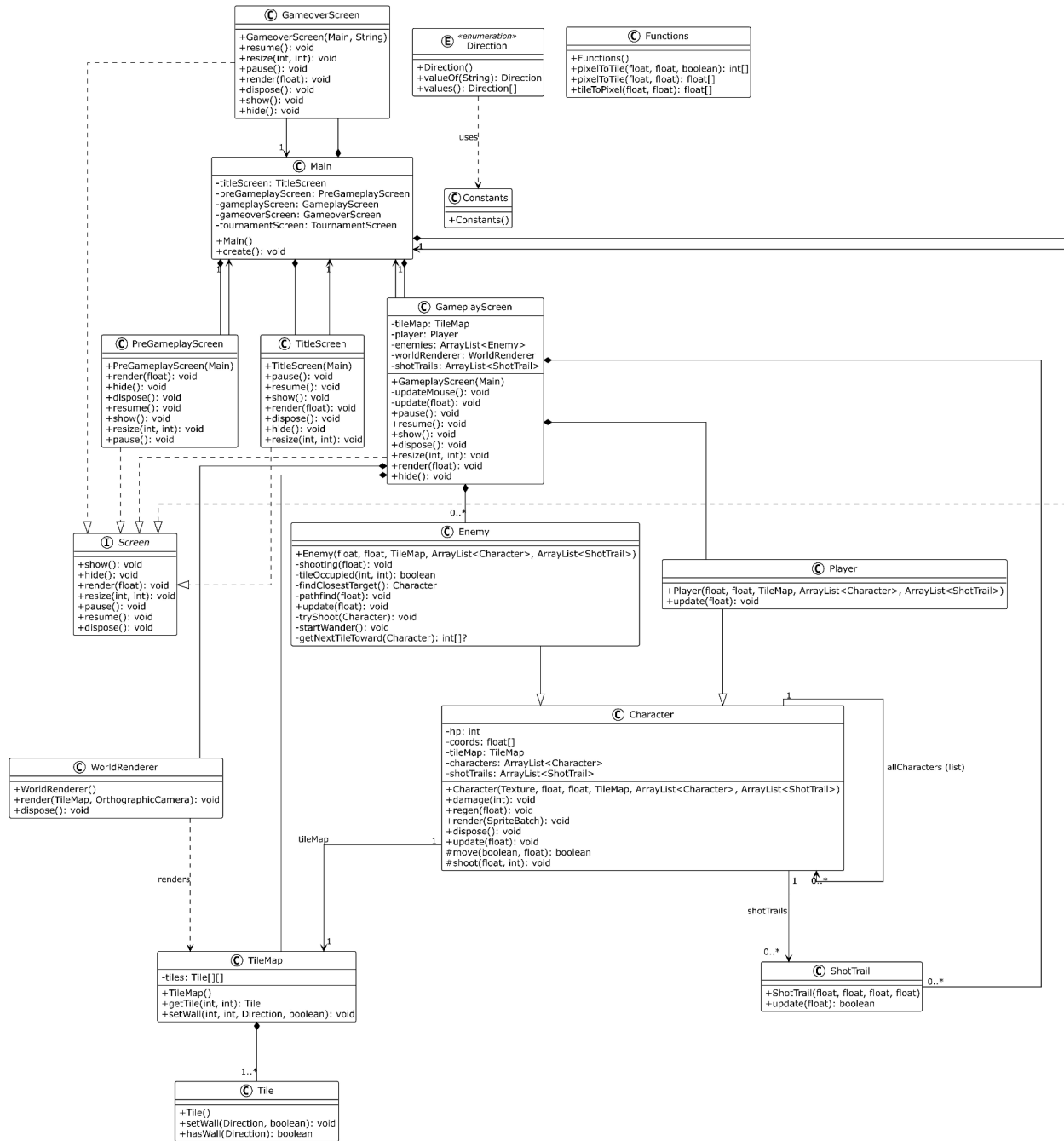
- Deliver a playable, real-time top-down shooter with procedurally generated maps, functional collision detection, enemy AI, and hitscan shooting mechanics.
- Implement a configurable pre-game setup screen that allows the player to adjust map size, player count, and enemy accuracy before each session.
- Build a tournament tracker as a separate screen within the same application with the ability to add players, record game results, sort and filter records, and predict match outcomes.

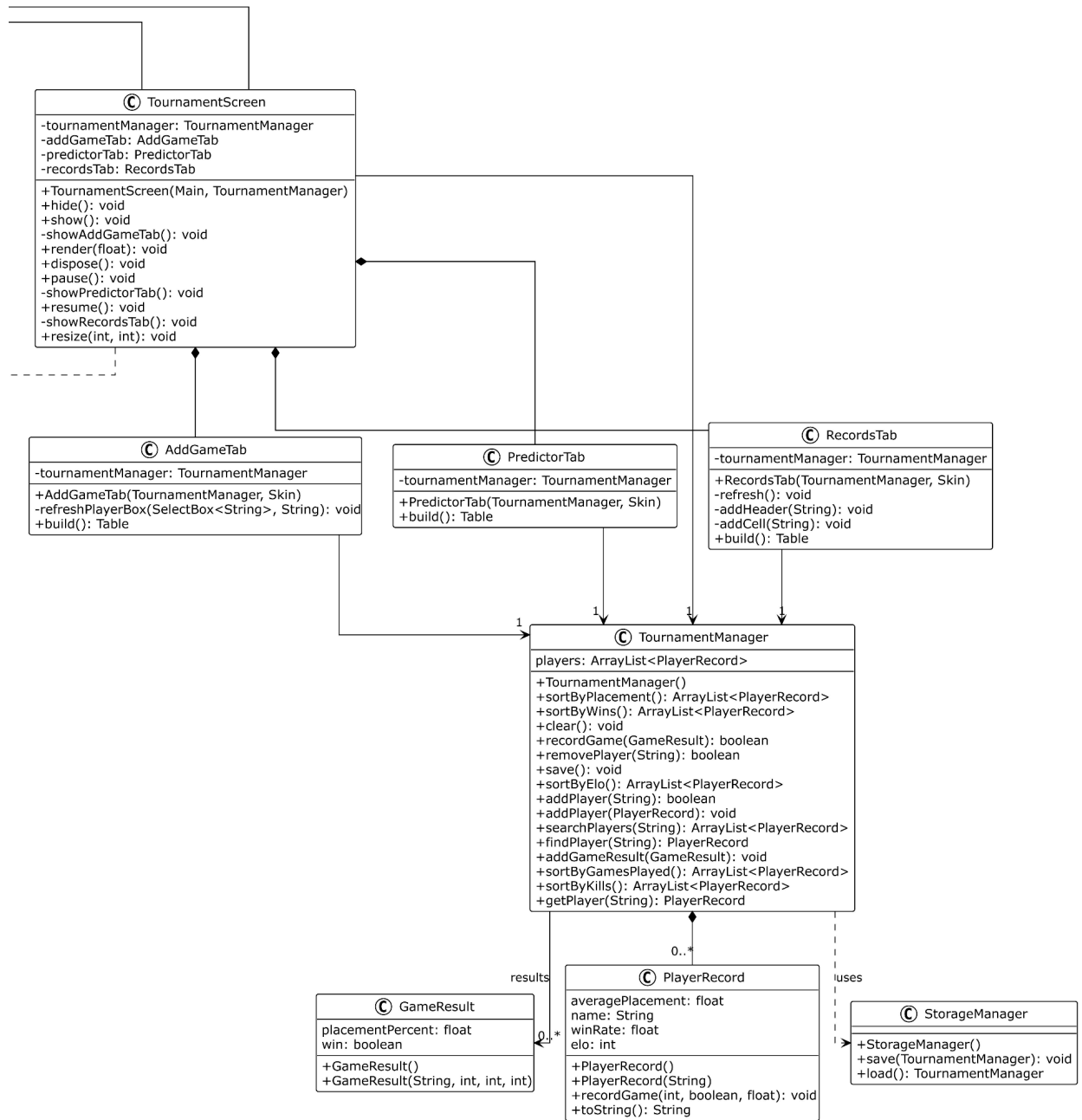
Program Architecture

The application (both the game and tracker) is made to be object-oriented from the ground up with Gradle + libGDX. The program is split into two, with the game and tournament tracker working independently from each other (only sharing the libGDX parts).

For the game, the entities in-game are part of the same superclass, where the subclasses override default behavior if needed. The world and its components (including tiles and bullet trails) are also separate objects.

The class diagram for the two parts of the program can be found below (split into 2 pages):





User Interface Design

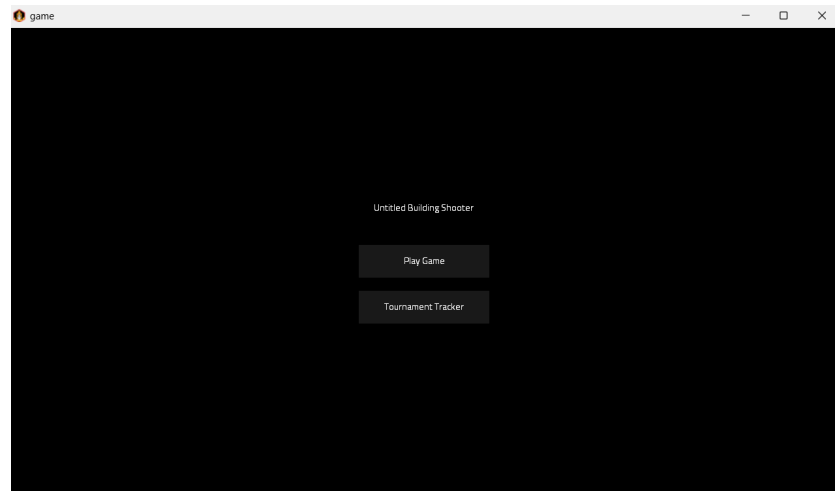


Fig A1:

This is the title screen, where you are given 2 buttons, the first being “Play Game”, which brings you into the game setup screen. The other button being “Tournament Tracker”, which brings you to the tournament tracker.

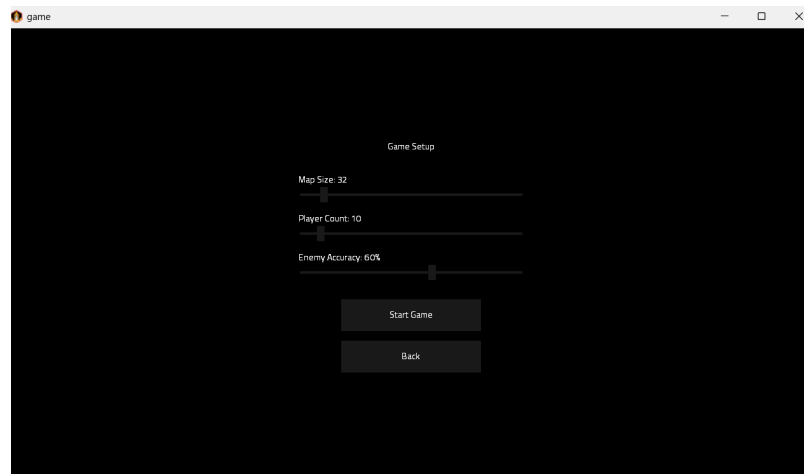


Fig A2:

This is the game setup screen, where we utilize sliders to change the settings of the match you are about to play in the game. There are 3 sliders present to change map size, player count and enemy accuracy. Then there are 2 buttons again, which are “Start Game”, which loads you straight into the game and “Back”, which takes you back to the title screen.

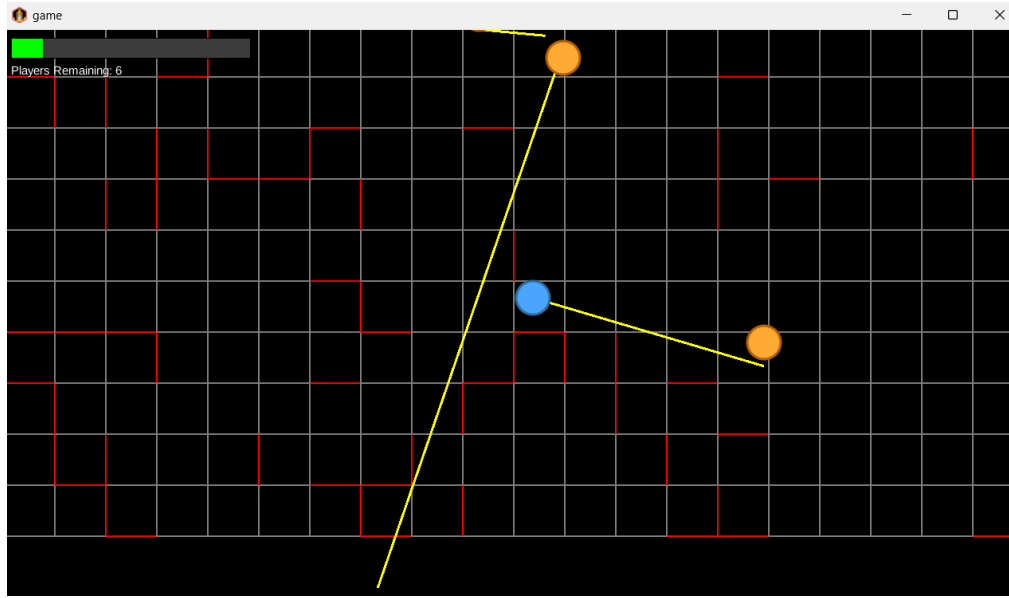


Fig A3:

This is the game itself, it features a health bar on the top left, which shows how much hit points you currently have, as well as it displays how many players are left remaining below it. A blue ball is used to indicate your player model, while enemy player models are colored orange. The yellow lines seen in the picture represents the bullet trails and the red lines are the walls you are able to build.

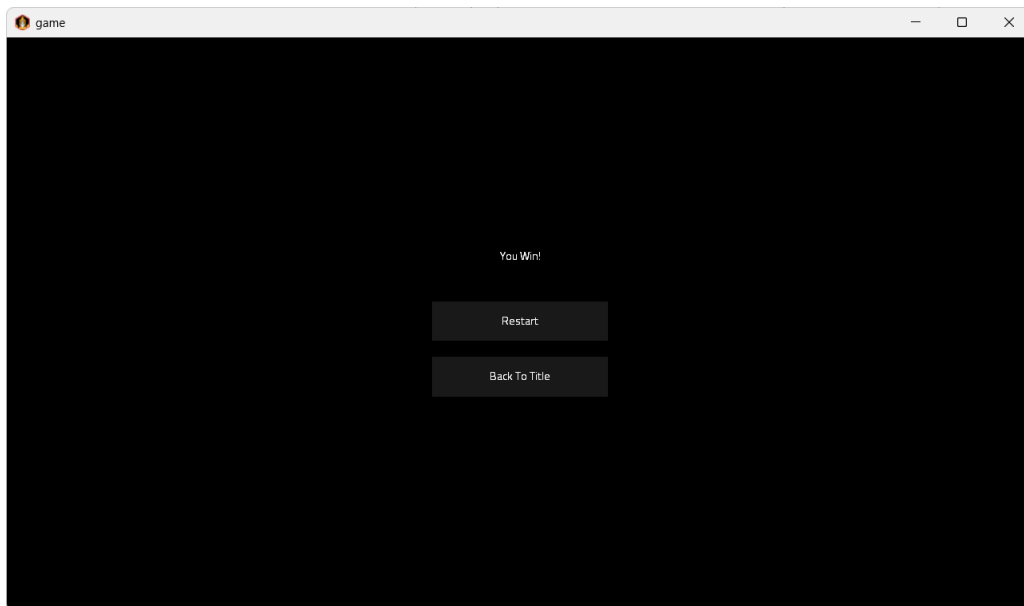


Fig A4:

This is the win end screen, which shows if you successfully become the last player standing. You are once again given 2 buttons which are "Restart" to play the same match again, and "Back To Title" to go back to the title screen.

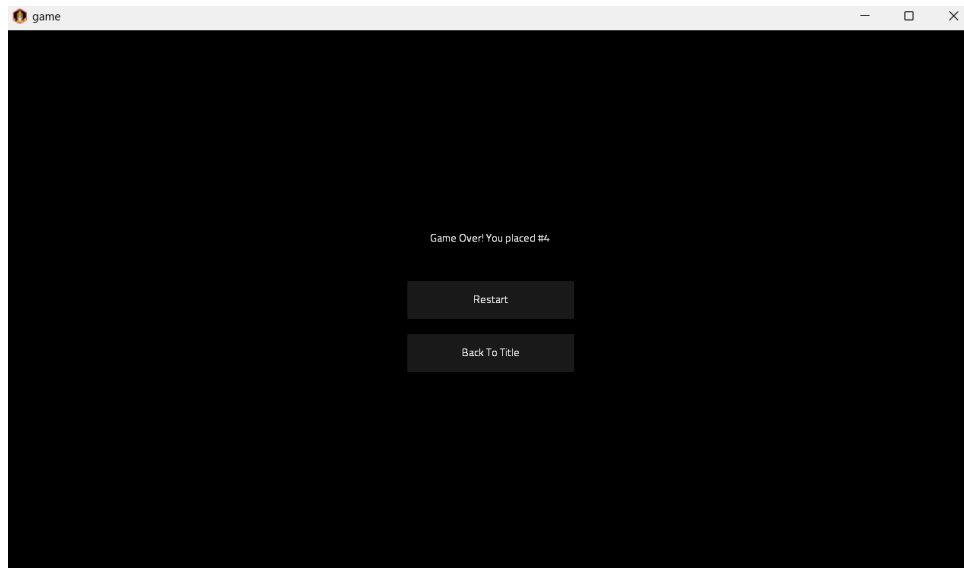


Fig A5:

Similar to the win end screen, this is the game over screen. The same exact thing seen in Fig A4, but this pops up if you are eliminated.

game

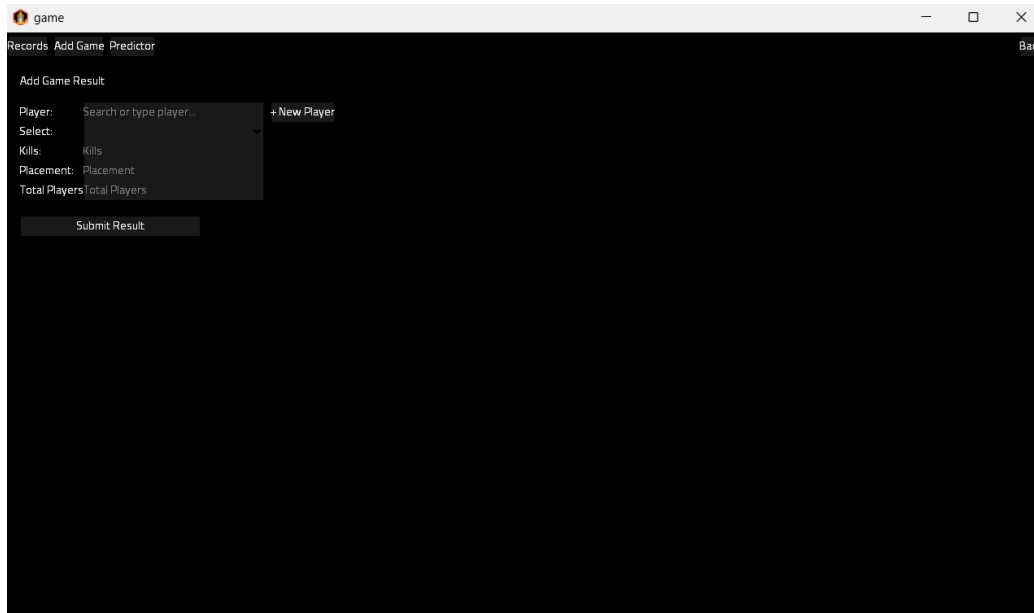
Records Add Game Predictor Back

Search: search player... Sort: Name Descending < Prev Page 1 / 10 Next >

Player	Kills	Games	Wins	Win %	Avg Place	ELO
Player99	2667	394	330	83.8%	0.6%	10267
Player98	3439	468	180	38.5%	0.6%	8039
Player97	12	9	2	22.2%	0.5%	1052
Player96	1004	335	86	25.7%	0.7%	3724
Player95	1022	181	128	70.7%	0.5%	4582
Player94	843	204	15	7.4%	0.9%	2143
Player93	1120	293	18	6.1%	0.8%	2480
Player92	329	161	64	39.8%	0.5%	2609
Player91	610	111	110	99.1%	0.5%	3810
Player90	1174	436	168	38.5%	0.9%	5534

Fig A6:

This is the tournament tracker, where you can see all the statistics and data of every single match that has happened.



Flg A7:

This is a game result adder, this is used to add new data such as a new game result or a new player.

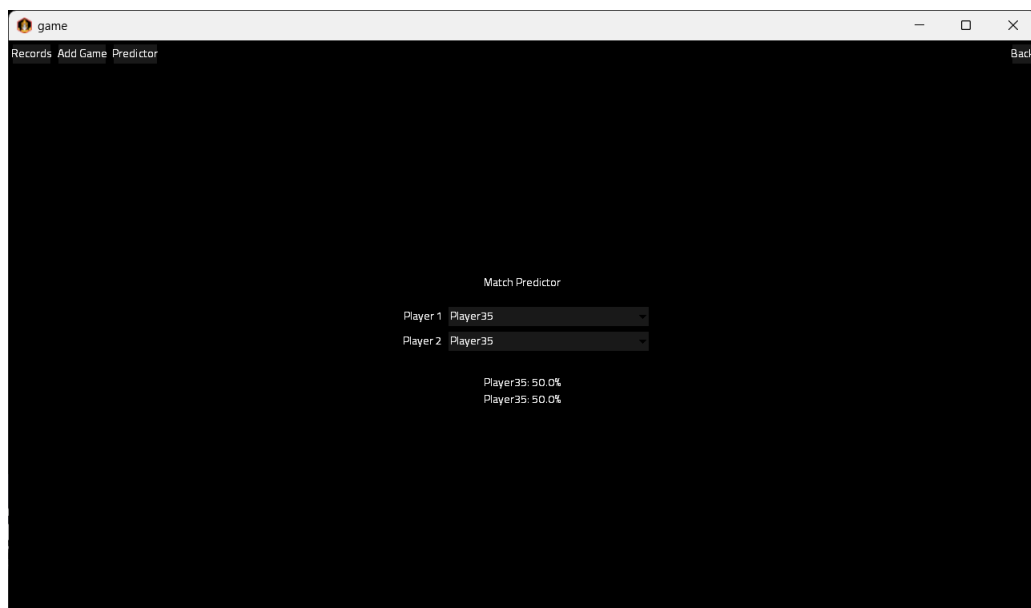


Fig A8:

This is the match predictor, where you are able to compare the winrates, if 2 players are put into a 1v1 situation. It displays the winrate in percentages.

Conclusion

This project was built for teamwork in mind, with its object oriented design and separation into multiple files to avoid git merge conflicts. The UI was also designed to be simple and user-friendly, as gamers need more time to learn the game rather than the UI.

Solution - Main Algorithms

```
public void show() {
    stage = new Stage(new StretchViewport(Constants.WIDTH, Constants.HEIGHT));
    Gdx.input.setInputProcessor(stage);

    skin = new Skin(Gdx.files.internal("ui/uiskin.json"));

    // layout stuff
    Table table = new Table();
    table.setFillParent(true);
    stage.addActor(table);

    Label title = new Label("Untitled Building Shooter", skin);
    TextButton playButton = new TextButton("Play Game", skin);
    TextButton tournamentButton = new TextButton("Tournament Tracker", skin);

    // play button
    playButton.addListener(new ClickListener() {
        @Override
        public void clicked(InputEvent event, float x, float y) {
            game.setScreen(new PreGameplayScreen(game));
        }
    });

    // tournament tracker
    tournamentButton.addListener(new ClickListener() {
        @Override
        public void clicked(InputEvent event, float x, float y) {

            // load saved tournament data
            TournamentManager manager = StorageManager.load();

            // open tracker
            game.setScreen(new TournamentScreen(game, manager));
        }
    });
    table.add(title).padBottom(50).row();
    table.add(playButton).padBottom(20).width(200).height(50).row();
    table.add(tournamentButton).width(200).height(50);
}
```

Fig B1: Title screen (Main)

The above code builds the title screen. Most of the code is libGDX layout code. The play and tournament buttons change the screen to the respective screens. For the tournament tracker screen, a TournamentManager object is first created so that the tracker has a single manager that it could “talk to”.

```

public void show() {
    camera = new OrthographicCamera();
    // separate cam for ui elements so it doesnt follow the world/game cam
    uiCam = new OrthographicCamera(Constants.WIDTH, Constants.HEIGHT);
    uiCam.position.set(0, 0, 0);

    viewport = new StretchViewport(Constants.WIDTH, Constants.HEIGHT, camera);
    font = new BitmapFont();

    viewport.apply();
    batch = new SpriteBatch();

    world = new TileMap();
    worldRenderer = new WorldRenderer();
    shapeRenderer = new ShapeRenderer();
    trails = new ArrayList<>();

    // populate entity list
    entities = new ArrayList<>();
    float x = (float) (Math.random() * Constants.MAP_SIZE * 64);
    float y = (float) (Math.random() * Constants.MAP_SIZE * 64);
    player = new Player(x, y, world, entities, trails);
    entities.add(player);
    for (int i = 0; i < Constants.ENEMY_COUNT; i++) {
        x = (float) (Math.random() * Constants.MAP_SIZE * 64);
        y = (float) (Math.random() * Constants.MAP_SIZE * 64);

        entities.add(new Enemy(x, y, world, entities, trails));
    }
}

```

Fig B2: Game Start Code

The above code initializes the game. It first initializes 2 cameras, as the game world and the UI are technically separate “layers” that move independently. The entire game is stretched to 1280x720. After the initial rendering stuff initializes, the world and its subsequent renderers are initialized. The entity list is an ArrayList consisting of Character objects (which includes Player and Enemy). These entities are then spawned with random coordinates within the world’s boundaries.

```

// render all
worldRenderer.render(world, camera);
shapeRenderer.setProjectionMatrix(camera.combined);
shapeRenderer.begin(ShapeRenderer.ShapeType.Filled);
for (int i = 0; i < trails.size(); i++) {
    ShotTrail t = trails.get(i);
    shapeRenderer.setColor(1f, 1f, 0f, t.life / 0.15f);
    shapeRenderer.rectLine(t.x1, t.y1, t.x2, t.y2, 3f);
}
shapeRenderer.end();

batch.setProjectionMatrix(camera.combined);
batch.begin();

// render players
for (int i = 0; i < entities.size(); i++) {
    Character c = entities.get(i);
    c.render(batch);
}

```

Fig B3: Rendering Order

The above code is part of the `GameplayScreen render()` function, and it takes place before rendering the UI. The world is rendered first, then players, and then the bullet trails, as seen by the `ShotTrail` renders toward the bottom. The players (`Character` objects) are rendered above everything else.

```

// left mouse click shooting
if (Gdx.input.isButtonJustPressed(0)) {
    float[] p = player.getCoords();

    // mouse - player center
    float dx = mousePos.x - (p[0] + 24);
    float dy = mousePos.y - (p[1] + 24);

    float angle = (float) Math.atan2(dy, dx);

    player.shoot(angle, 10);
}

// right click wall building
if (Gdx.input.isButtonJustPressed(1)) {
    float[] p = player.getCoords();

    // mouse - player center
    float dx = mousePos.x - (p[0] + 24);
    float dy = mousePos.y - (p[1] + 24);

    // player tile
    int[] tile = Functions.pixelToTile(p[0], p[1], true);
    Constants.Direction dir;

    // choose dominant axis
    if (Math.abs(dx) > Math.abs(dy)) {
        if (dx > 0) {
            dir = Constants.Direction.EAST;
        } else {
            dir = Constants.Direction.WEST;
        }
    } else {
        if (dy > 0) {
            dir = Constants.Direction.NORTH;
        } else {
            dir = Constants.Direction.SOUTH;
        }
    }

    // place wall
    world.setWall(tile[0], tile[1], dir, true);
}

```

Fig B4: Mouse Handling

The above code is responsible for handling the mouse button actions in the game. Left click for shooting, and right click for building. For shooting, the mouse's angle is calculated by using a geometry formula with the differences between the mouse and player positions. The same formulas are used for building, but the most dominant direction (determined by combinations of whether dx or dy are larger) is chosen instead of needing to use a proper angle. This is done to increase performance.

```

protected boolean move(boolean isVertical, float amount) {
    float newX = x;
    float newY = y;

    if (isVertical) {
        newY += amount;
    } else {
        newX += amount;
    }

    // for collision checking
    float left = newX;
    float right = newX + 48;
    float bottom = newY;
    float top = newY + 48;

    // left
    if (!isVertical && amount < 0) {
        int currentTileX = (int) Math.floor(x / 64f);
        int newTileX = (int) Math.floor(left / 64f);

        int bottomTile = (int) Math.floor(bottom / 64f);
        int topTile = (int) Math.floor((top - 1) / 64f);

        // if tile bound crossed
        if (newTileX < currentTileX) {
            for (int ty = bottomTile; ty <= topTile; ty++) {
                Tile tile = world.getTile(currentTileX, ty);
                // dont move further if wall exists between boundary cross
                if (tile != null && tile.hasWall(Constants.Direction.WEST)) {
                    return false;
                }
            }
        }
    }

    // right
    if (!isVertical && amount > 0) {
        int currentTileX = (int) Math.floor((x + 47) / 64f);
        int newTileX = (int) Math.floor(right / 64f);

        int bottomTile = (int) Math.floor(bottom / 64f);
        int topTile = (int) Math.floor((top - 1) / 64f);

        if (newTileX > currentTileX) {
            for (int ty = bottomTile; ty <= topTile; ty++) {
                Tile tile = world.getTile(currentTileX, ty);
                if (tile != null && tile.hasWall(Constants.Direction.EAST)) {
                    return false;
                }
            }
        }
    }

    // down
    if (isVertical && amount < 0) {
        int currentTileY = (int) Math.floor(y / 64f);
        int newTileY = (int) Math.floor(bottom / 64f);

        int leftTile = (int) Math.floor(left / 64f);
        int rightTile = (int) Math.floor((right - 1) / 64f);

        if (newTileY < currentTileY) {
            for (int tx = leftTile; tx <= rightTile; tx++) {
                Tile tile = world.getTile(tx, currentTileY);
                if (tile != null && tile.hasWall(Constants.Direction.SOUTH)) {
                    return false;
                }
            }
        }
    }

    // up
    if (isVertical && amount > 0) {
        int currentTileY = (int) Math.floor((y + 47) / 64f);
        int newTileY = (int) Math.floor(top / 64f);

        int leftTile = (int) Math.floor(left / 64f);
        int rightTile = (int) Math.floor((right - 1) / 64f);

        if (newTileY > currentTileY) {
            for (int tx = leftTile; tx <= rightTile; tx++) {
                Tile tile = world.getTile(tx, currentTileY);
                if (tile != null && tile.hasWall(Constants.Direction.NORTH)) {
                    return false;
                }
            }
        }
    }

    // only goes here if the movement didnt collide with wall (edge of world is
    // handled differently)
    x = newX;
    y = newY;
    x = Math.clamp(x, 0f, (Constants.MAP_SIZE - 1) * 64 + Constants.PX_LEFTOVER);
    y = Math.clamp(y, 0f, (Constants.MAP_SIZE - 1) * 64 + Constants.PX_LEFTOVER);
    return true;
}

```

Fig B5: Character Move Function

The above code is the move function as part of the Character abstract class (which the Player and Enemy classes inherit from). Before any real movement or checks are done, the new position (and corresponding hitbox bounds) are predicted. These predicted coordinates are then checked against any existing walls (if a tile boundary is crossed during the move). If any tile boundaries are crossed, the movement is simply canceled. If all checks pass, the movement is finally written to the current object's coordinates, and the coordinates are then clamped so the character can't move out of the world's boundaries.


```

// raycast algo for hitscan
protected void shoot(float angle, int damage) {
    final float STEP = 4f; // resolution

    // ray start coords and angle mults (yay trig)
    float px = x + 24;
    float py = y + 24;
    float dx = (float) Math.cos(angle);
    float dy = (float) Math.sin(angle);

    // ray rendering
    float startX = px;
    float startY = py;

    int[] prevTile = Functions.pixelToTile(px, py, false);
    int prevTileX = prevTile[0];
    int prevTileY = prevTile[1];

    // keep casting until break
    while (true) {
        // move ray
        px += dx * STEP;
        py += dy * STEP;

        // current tile ray is in
        int[] tilecoords = Functions.pixelToTile(px, py, false);
        int tileX = tilecoords[0];
        int tileY = tilecoords[1];

        // ray oob
        if (tileX < 0 || tileY < 0 || tileX >= Constants.MAP_SIZE || tileY >= Constants.MAP_SIZE) {
            trails.add(new ShotTrail(startX, startY, px, py));
            return;
        }

        // check for walls during tile boundary crosses
        Tile tile = world.getFile(prevTileX, prevTileY);
        if (tile != null) {
            // east wall
            if (tileX > prevTileX && tile.hasWall(Constants.Direction.EAST)) {
                world.setWall(prevTileX, prevTileY, Constants.Direction.EAST, false);
                trails.add(new ShotTrail(startX, startY, px, py));
                return;
            }
            // west wall
            if (tileX < prevTileX && tile.hasWall(Constants.Direction.WEST)) {
                world.setWall(prevTileX, prevTileY, Constants.Direction.WEST, false);
                trails.add(new ShotTrail(startX, startY, px, py));
                return;
            }
            // north wall
            if (tileY > prevTileY && tile.hasWall(Constants.Direction.NORTH)) {
                world.setWall(prevTileX, prevTileY, Constants.Direction.NORTH, false);
                trails.add(new ShotTrail(startX, startY, px, py));
                return;
            }
            // south wall
            if (tileY < prevTileY && tile.hasWall(Constants.Direction.SOUTH)) {
                world.setWall(prevTileX, prevTileY, Constants.Direction.SOUTH, false);
                trails.add(new ShotTrail(startX, startY, px, py));
                return;
            }
        }

        // character hit
        for (int i = 0; i < entities.size(); i++) {
            Character c = entities.get(i);
            // ignore self
            if (c == this) {
                continue;
            }
            // check if ray is within 48x48px hitbox
            if (px >= c.x && px <= c.x + 48 && py >= c.y && py <= c.y + 48) {
                c.damage(damage);
                trails.add(new ShotTrail(startX, startY, px, py));
                return;
            }
        }

        // prepare for next iteration
        prevTileX = tileX;
        prevTileY = tileY;
    }
}

```

Fig B6: Character Shoot Function (Raycast)

The above code is responsible for any shooting the players in the game do. It uses a raycasting algorithm. The raycaster uses steps, so that it isn't too intensive on the CPU. I found that a resolution/step of 4 pixels is more than enough to find collisions with walls or players. It then uses a similar algorithm to the movement collision function to cut the ray short. When the raycaster hits a wall, it creates a new ShotTrail with the last "good" coordinates.

```
public void damage(int damage) {
    this.hp -= damage;
    // remove self if dead
    if (this.hp <= 0) {
        entities.remove(this);
        dispose();
    }
}

public void regen(float dt) {
    // dead entities dont regen
    if (hp <= 0) {
        return;
    }

    regenTimer += dt;
    if (regenTimer >= Constants.REGEN_INTERVAL) {
        hp++;
        regenTimer = 0f;
    }
    hp = Math.clamp(hp, 0, 100);
}
```

Fig B7: Damage and regeneration functions

The above code includes wrappers for the player's health. As there is no scenario for the player to regenerate rapidly, only small increments of regenerations are allowed, while damage can be instant. If the health is 0 or lower, the current object is directly removed from the entity list. As for regeneration, it regenerates every REGEN_INTERVAL (which is adjustable). Every time this timer is hit, the health increments slightly, and the timer resets. The delta time is used so that the regeneration remains consistent regardless of framerate (unless frametimes get higher than the interval).

```
public void update(float dt) {
    regen(dt);

    // movement is handled by keyboard instead of automatic pathfind
    if (Gdx.input.isKeyPressed(Input.Keys.W)) {
        move(true, (speed * dt));
    }
    if (Gdx.input.isKeyPressed(Input.Keys.S)) {
        move(true, -(speed * dt));
    }

    if (Gdx.input.isKeyPressed(Input.Keys.A)) {
        move(false, -(speed * dt));
    }

    if (Gdx.input.isKeyPressed(Input.Keys.D)) {
        move(false, (speed * dt));
    }
}
```

Fig B8: Player Update Function

This is the update function for the Player class. It only regenerates and calls the movement wrapper when WASD is pressed.

```

// behold, pathfinding :tired:
private void pathfind(float dt) {
    // wander mode handler
    if (wanderTimer > 0f) {
        // decrement timers
        wanderTimer -= dt;
        wanderChangeTimer -= dt;

        // pick direction once every ~0.2s OR if blocked
        if (wanderChangeTimer <= 0f) {
            wanderDir = random.nextInt(4);
            wanderChangeTimer = 0.2f;
        }

        boolean moved = false;

        // move in a semi-random direction
        switch (wanderDir) {
            case 0:
                moved = move(true, speed * dt);
                break;
            case 1:
                moved = move(true, -speed * dt);
                break;
            case 2:
                moved = move(false, speed * dt);
                break;
            case 3:
                moved = move(false, -speed * dt);
                break;
        }

        // if blocked, force immediate reroll next frame
        if (!moved) {
            wanderChangeTimer = 0f;
        }

        return;
    }

    // find next target to pathfind to
    Character target = findClosestTarget();
    if (target == null) {
        return;
    }

    // find next tile to move to
    int[] nextTile = getNextTileToward(target);
    if (nextTile == null) {
        return;
    }

    // someone already heading/standing there?
    if (tileOccupied(nextTile[0], nextTile[1])) {
        startWander();
        return;
    }

    // run away if hp is low
    if (getHp() <= 20) {
        startWander();
        return;
    }

    // next pixel target to move toward [x,y]
    float[] nextPixel = Functions.tileToPixel(nextTile[0], nextTile[1]);
    float targetX = nextPixel[0] + 8f;
    float targetY = nextPixel[1] + 8f;

    // distances
    float dx = targetX - x;
    float dy = targetY - y;

    // do the movement
    if (Math.abs(dx) > 2f) {
        move(false, Math.signum(dx) * speed * dt);
    }
    if (Math.abs(dy) > 2f) {
        move(true, Math.signum(dy) * speed * dt);
    }
}

```

Fig B9: Enemy Pathfinding Loop

The above function is the code responsible for the movement logic loop in the Enemy class AI. Before any kind of movement is done, it checks if wander mode is active. If it is, the Enemy moves in a random direction. Otherwise, it calls findClosestTarget and getNextTileToward to BFS to the closest Character. If the tile it's going to is occupied or the health is low, it wanders. Then, it does the actual movement with the movement wrapper from the parent Character class.

```
private Character findClosestTarget() {
    Character closest = null;
    float bestDist = Float.MAX_VALUE;

    for (int i = 0; i < entities.size(); i++) {
        Character c = entities.get(i);
        // integrity check
        if (c == this) {
            continue;
        }

        // basic math formula but skip the sqrt
        float dx = c.x - x;
        float dy = c.y - y;
        float dist = dx * dx + dy * dy;

        if (dist < bestDist) {
            bestDist = dist;
            closest = c;
        }
    }

    return closest;
}
```

Fig B10: Find Closest Target Function

The above code is referred to in Figure B9, where an Enemy would constantly try to find the closest target to travel to. This function handles that. It simply does a distance check with every single item in the entities list, and returns the smallest distance Character.

```

// bfs search for next tile
private int[] getNextTileToward(Character target) {
    int[] start = Functions.pixelToTile(x, y, true);
    int[] goal = Functions.pixelToTile(target.x, target.y, true);

    // aliases
    int startX = Math.clamp(start[0], 0, Constants.MAP_SIZE - 1);
    int startY = Math.clamp(start[1], 0, Constants.MAP_SIZE - 1);
    int goalX = Math.clamp(goal[0], 0, Constants.MAP_SIZE - 1);
    int goalY = Math.clamp(goal[1], 0, Constants.MAP_SIZE - 1);

    // already there
    if (startX == goalX && startY == goalY) {
        return null;
    }

    boolean[][] visited = new boolean[Constants.MAP_SIZE][Constants.MAP_SIZE];
    int[][] parentX = new int[Constants.MAP_SIZE][Constants.MAP_SIZE];
    int[][] parentY = new int[Constants.MAP_SIZE][Constants.MAP_SIZE];

    Queue<int[]> queue = new LinkedList<>();

    // start BFS here
    queue.add(new int[] { startX, startY });
    visited[startX][startY] = true;

    // aliases for cardinal directions
    int[][] dirs = {
        { 0, 1 },
        { 0, -1 },
        { 1, 0 },
        { -1, 0 }
    };

    // main bfs loop
    while (!queue.isEmpty()) {
        int[] cur = queue.poll();

        int cx = cur[0];
        int cy = cur[1];

        // stop if goal reached
        if (cx == goalX && cy == goalY) {
            break;
        }

        Tile tile = world.getTile(cx, cy);

        // visit all cardinal directions
        for (int i = 0; i < 4; i++) {
            // weird edge case
            if (tile == null) {
                continue;
            }
            // visited tile coords
            int nx = cx + dirs[i][0];
            int ny = cy + dirs[i][1];

            // skip out of bounds and visited
            if (nx < 0 || ny < 0 || nx >= Constants.MAP_SIZE || ny >= Constants.MAP_SIZE) {
                continue;
            }
            if (visited[nx][ny]) {
                continue;
            }

            // check for wall
            boolean blocked = false;
            switch (i) {
                case 0:
                    blocked = tile.hasWall(Constants.Direction.NORTH);
                    break;
                case 1:
                    blocked = tile.hasWall(Constants.Direction.SOUTH);
                    break;
                case 2:
                    blocked = tile.hasWall(Constants.Direction.EAST);
                    break;
                case 3:
                    blocked = tile.hasWall(Constants.Direction.WEST);
                    break;
            }

            // cancel visit if theres a wall
            if (blocked) {
                continue;
            }

            // mark visited and mark BFS'd path
            visited[nx][ny] = true;
            parentX[nx][ny] = cx;
            parentY[nx][ny] = cy;

            // redo BFS
            queue.add(new int[] { nx, ny });
        }
    }

    // no path found
    if (!visited[goalX][goalY]) {
        return null;
    }

    // reconstruct path
    // current tile (work backwards)
    int cx = goalX;
    int cy = goalY;

    // walk back from the previously logged parent paths
    while (!(parentX[cx][cy] == startX && parentY[cx][cy] == startY)) {
        int px = parentX[cx][cy];
        int py = parentY[cx][cy];

        // integrity check
        if (px == cx && py == cy) {
            return null;
        }

        // set for next iter
        cx = px;
        cy = py;
    }

    return new int[] { cx, cy };
}

```

Fig B11: Enemy AI BFS Function

The function above is responsible for pathfinding. It uses a breadth-first search (as seen by the LinkedList queue). It first defines the start coordinates as the current Character position, and it defines end coordinates as the position of the Character it's trying to pathfind to. It then does a breadth-first search (enqueue all adjacent nodes and visit queued nodes). By default, all adjacent tiles can be traveled to unless there is a wall. It then keeps track of the traveled path by keeping track of where each tile came from in the path using the parentX and parentY arrays. It then walks back up that array to figure out the next tile to travel to.

```
public Tile getTile(int x, int y) {
    // bounds check
    if (x < 0 || y < 0 || x >= MAP_SIZE || y >= MAP_SIZE) {
        return null;
    }
    return tiles[x][y];
}

public void setWall(int x, int y, Constants.Direction dir, boolean value) {
    Tile tile = getTile(x, y);
    // integrity check
    if (tile == null) {
        return;
    }

    tile.setWall(dir, value);
    // set adjacent wall's tile too, since walls are shared between tiles
    switch (dir) {
        case NORTH:
            Tile north = getTile(x, y + 1);
            if (north != null)
                north.setWall(Constants.Direction.SOUTH, value);
            break;
        case SOUTH:
            Tile south = getTile(x, y - 1);
            if (south != null)
                south.setWall(Constants.Direction.NORTH, value);
            break;
        case EAST:
            Tile east = getTile(x + 1, y);
            if (east != null)
                east.setWall(Constants.Direction.WEST, value);
            break;
        case WEST:
            Tile west = getTile(x - 1, y);
            if (west != null)
                west.setWall(Constants.Direction.EAST, value);
            break;
    }
}
```

Fig 12B: TileMap Wrappers

The above functions are part of the TileMap (world) object, used as setters and getters for the walls. The getTile function has basic boundary checks so that out-of-bounds tiles aren't accessed. As for the setWall function, walls in adjacent tiles are set because adjacent tiles share one wall. So, the north wall of a tile is equivalent to the south wall of the tile to its north.

```
// add player by name
public boolean addPlayer(String name) {

    String key = name.toLowerCase();

    if (players.containsKey(key)) {
        return false;
    }

    players.put(key, new PlayerRecord(name));
    return true;
}

// overload for AddGameTab
public void addPlayer(PlayerRecord player) {
    players.put(player.getName().toLowerCase(), player);
}

public boolean removePlayer(String name) {

    String key = name.toLowerCase();

    if (!players.containsKey(key)) {
        return false;
    }

    players.remove(key);
    return true;
}
```

Fig 13B: Add Player Function

```
// search/filter for RecordsTab
public ArrayList<PlayerRecord> searchPlayers(String query) {

    ArrayList<PlayerRecord> results = new ArrayList<>();

    if (query == null || query.isBlank()) {
        results.addAll(players.values());
        return results;
    }

    String q = query.toLowerCase();

    for (PlayerRecord p : players.values()) {
        if (p.name.toLowerCase().contains(q)) {
            results.add(p);
        }
    }

    return results;
}
```

Fig 14B: Search Players Function

The above functions (and their overloads) are part of the TournamentManager class. The first one adds a player to the players HashMap. The second one searches for players. It does a linear search, as it needs to find substrings within the player names instead of the entire player name. Unfortunately, this does tank performance.

```
public void recordGame(
    int kills,
    boolean won,
    float placementPercent) {

    this.kills += kills;
    this.gamesPlayed++;

    if (won) {
        this.wins++;
    }

    // running average update
    averagePlacement =
        (averagePlacement * placementGames
         + placementPercent)
        / (placementGames + 1);

    placementGames++;
}

// derived ELO formula
// replace this with whatever formula you want
public int getElo() {

    // placeholder formula
    return 1000 + wins * 20 + kills;
}
```

Fig 15B: Player Statistics Functions

The functions above are part of the PlayerRecord class, and they respectively add a game to the player's stats and calculate the player's ELO. The game is recorded by adding the total kills and incrementing the games played (and wins if the game was a win). The average placement is calculated using a running average (scaling the current game by a factor of the current amount of games played), which helps with performance as individual match records don't need to be stored.

```
// save manager data
public static void save(TournamentManager manager) {

    Json json = new Json();

    ArrayList<PlayerRecord> players = manager.getPlayers();

    String data = json.prettyPrint(players);

    FileHandle file = Gdx.files.local(SAVE_FILE);
    file.writeString(data, false);
}

// load manager data
@SuppressWarnings("unchecked")
public static TournamentManager load() {

    TournamentManager manager = new TournamentManager();
    FileHandle file = Gdx.files.local(SAVE_FILE);

    // first launch / no save
    if (!file.exists()) {
        return manager;
    }

    try {

        Json json = new Json();

        ArrayList<PlayerRecord> players =
            json.fromJson(ArrayList.class,
                PlayerRecord.class,
                file.readString());

        if (players != null) {
            manager.setPlayers(players);
        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    return manager;
}
```

Fig 16B: JSON Saving and Loading

```
// UI snapshot
public ArrayList<PlayerRecord> getPlayers() {
    return new ArrayList<>(players.values());
}
```

Fig 17B: Get Players as ArrayList

```
[
  {
    "name": "Player0",
    "kills": 3290,
    "gamesPlayed": 481,
    "wins": 71,
    "averagePlacement": 0.2547,
    "placementGames": 481
  },
  {
    "name": "Player1",
    "kills": 21,
    "gamesPlayed": 25,
    "wins": 2,
    "averagePlacement": 0.4955,
    "placementGames": 25
  },
  {
    "name": "Player2",
    "kills": 2417,
    "gamesPlayed": 357,
    "wins": 31,
    "averagePlacement": 0.4949,
    "placementGames": 357
  }
]
```

Fig 18B: Saved JSON

The functions above handle saving and loading to JSON. They use the libGDX JSON library. They simply get all players to/from the TournamentManager (which will return it as an ArrayList), and save/load accordingly.

Solution - Data Structures

Description and Analysis

For this project, 3 data structures were tried for both the tournament tracker and main game. For the tournament tracker, HashMap, TreeMap, and ArrayList were tried. For the main game's map, 2D Array, ArrayList, and TreeMap were tried.

HashMap

HashMap is the chosen data structure for the tournament tracker. HashMaps store key/value pairs, which makes it perfect to look up players. It has an average case of $O(1)$ for insertion, deletion, and lookups, which also makes it favorable for larger databases, such as the one found in the tournament tracker. The $O(1)$ average case works because the hash function is deterministic, and it allows an instant index access to the HashMap's internal data store.

TreeMap

A TreeMap is similar to a HashMap. However, it sorts the keys internally using a self-balancing binary search tree. This allows automatic sorting of player names, however it adds extra steps to adding or deleting. The tournament tracker already has its own sorting system using the Java collections sorting methods, so this becomes redundant, and lookup becomes $O(\log n)$.

ArrayList

An ArrayList is a wrapper class for a Java array. It simply stores the items the same way an array would, but it has extra functionality to be able to copy the items to a new, larger array if needed, and also to shift items left when something is deleted. While this has extremely fast accesses similar to a regular array, searching through one is $O(n)$, as there are no keys.

2D Array

A 2D array is just an array that contains arrays. They have static size, and every operation except for search is $O(1)$, as the indices are all pre-determined. This was chosen for ease of programming in the world tile map, as the abstraction from other data structures wasn't needed, and the world had a static size.

Comparison and Evaluation

Tournament Benchmark				TileMap Benchmark				
Players: 100				32x32				
DS	Insert (ms)	Lookup (ms)	Sort (ms)	DS	Construct (ms)	Lookup (ms)	Update (ms)	BFS (ms)
HashMap	2.411	0.0889	1.6067	2D Array	0.180465	19.960966	36.746196	0.315833
TreeMap	0.781	0.0544	0.2974	HashMap	12.394787	152.174644	304.446703	0.785409
ArrayList	3.5617	0.0993	0.3306	ArrayList	1.349095	0.001963	0.000911	0.012244
Players: 1000				128x128				
DS	Insert (ms)	Lookup (ms)	Sort (ms)	DS	Construct (ms)	Lookup (ms)	Update (ms)	BFS (ms)
HashMap	1.9287	0.0038	0.1619	2D Array	0.171835	20.141242	29.863831	0.383771
TreeMap	11.3048	0.006	0.2241	HashMap	12.302319	28.768965	47.17252	0.419923
ArrayList	348.5387	0.3454	0.1598	ArrayList	1.355899	0.002017	0.000684	0.010214
Players: 10000				256x256				
DS	Insert (ms)	Lookup (ms)	Sort (ms)	DS	Construct (ms)	Lookup (ms)	Update (ms)	BFS (ms)
HashMap	19.3813	0.0045	1.4072	2D Array	0.18405	19.676568	28.804693	0.794912
TreeMap	145.3849	0.0079	2.1318	HashMap	12.237987	21.90196	33.24711	0.821342
ArrayList	36328.4783	3.625	1.1247	ArrayList	1.365879	0.001888	0.000609	0.012114

Fig 1C: Performance Comparison Table

The above tables are the performance comparisons of the aforementioned data structures for both the tournament tracker and world tile map. As seen from the benchmark results, HashMap scales up the best for insert, lookup, and sorting compared to all the other data structures (such as ArrayList that takes 36 seconds to insert at 10,000 players). This was especially useful, as

the real program was tested to be able to handle 100,000 players. The growth rate of insertion (using linear regression) can be seen in this graph below:

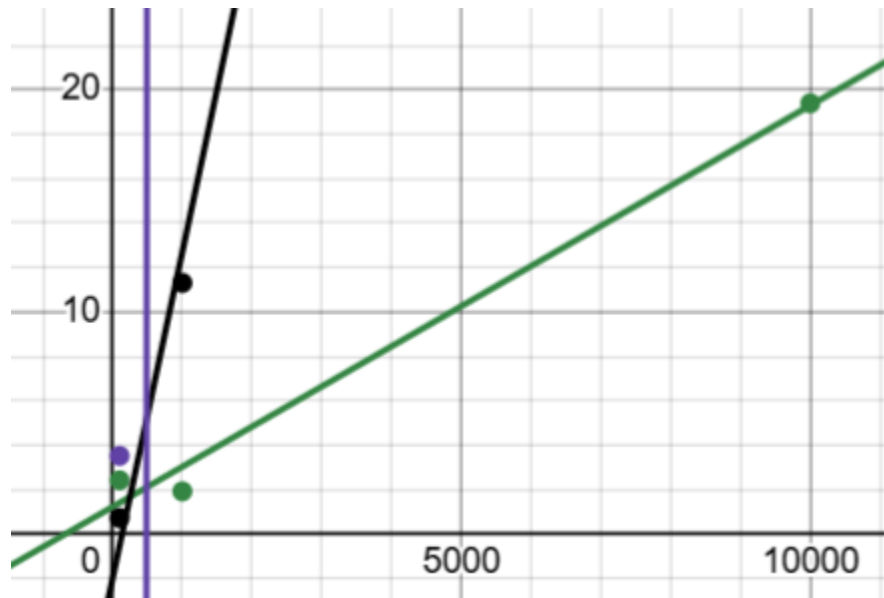


Fig 2C: Growth - Green: HashMap, Black: TreeMap, Purple: ArrayList

That alone was enough to consider using HashMap. The other operations all completed in a few milliseconds even when scaled up, so the difference there was negligible.

World sizes are small, so the choice for what data structure to use was clear. 2D arrays are the easiest to code for. Looking at the graph, the amount of time for all operations are approximately the same regardless of tested world sizes (the game only goes up to 256), with the exception of the BFS that suddenly takes twice as long at 256x256. The ArrayList times are also consistently low across the board, which is baffling. This could be due to many factors, such as measurement noise and/or inconsistencies such as JVM/JIT compilation warmup, as JIT compilers tend to cache functions. Furthermore, ArrayLists are part Java, which means that the JVM likely has optimized implementations of it already for all platforms, as opposed to needing to “interpret” a standard 2D array. Furthermore, the small world sizes and negligible growth suggest that the tested data structures aren’t big enough to fully test performance, as the test includes the algorithms that exist in the program already, which may take significantly more time than it takes to do raw accesses to the data structure itself.

References

Agrawal, A. (n.d.). Data science and analytics for esports. Johns Hopkins University Institute for Data Intensive Engineering and Science.

<https://www.idies.jhu.edu/wp-content/uploads/2021/10/agrawal.pdf>

CEUR Workshop Proceedings. (2015). Proceedings of GSB 2015 (p. 15). CEUR Workshop Proceedings. <https://ceur-ws.org/Vol-1393/gsb15proceedings.pdf>

Cui, X. (n.d.). A pathfinding in modern computer games. ResearchGate.
https://www.researchgate.net/profile/Xiao-Cui-12/publication/267809499_A-based_Pathfinding_in_Modern_Computer_Games/links/54fd73740cf270426d125adc/A-based-Pathfinding-in-Modern-Computer-Games.pdf

GeeksforGeeks. (2012). Breadth First Search or BFS for a Graph. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>

GeeksforGeeks. (2016). Array vs ArrayList in Java. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/array-vs-arraylist-in-java/>

GeeksforGeeks. (2018). TreeMap in Java. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/treemap-in-java/>

Louisiana Tech University. (n.d.). Mathematics senior capstone papers. Louisiana Tech University Digital Commons.
<https://digitalcommons.latech.edu/cgi/viewcontent.cgi?article=1010&context=mathematics-senior-capstone-papers>

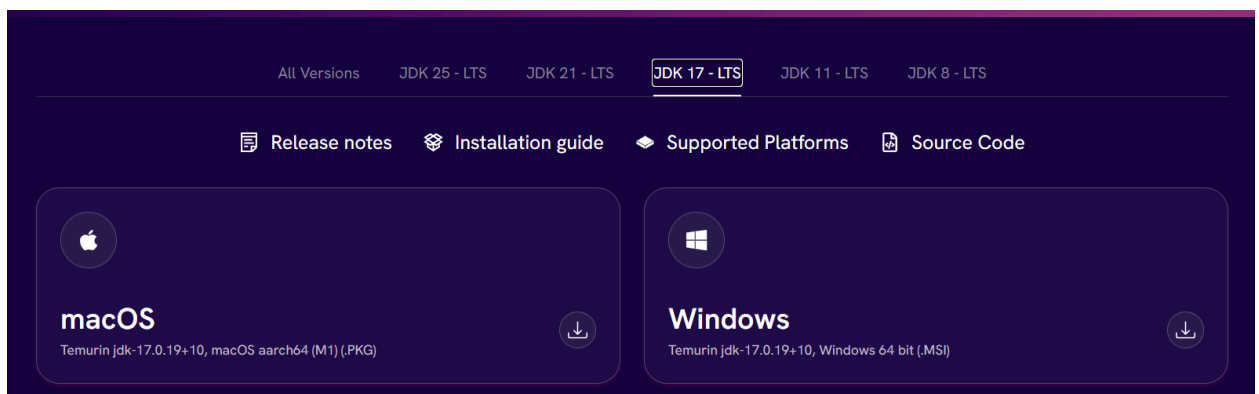
MDPI. (2024). Algorithms, 17(9), 404. <https://www.mdpi.com/1999-4893/17/9/404>

Springer. (n.d.). Springer. https://link.springer.com/chapter/10.1007/978-1-4612-4400-4_4

Winona State University. (2016). CS conference proceedings 2016 (p. 35). Winona State University.
https://cs.winona.edu/cs-website/current_students/Projects/CSConference/2016conference.pdf

Appendix - Build Guide (Windows)

1. Download Temurin JDK 17



2. Clone or download the git repository

```

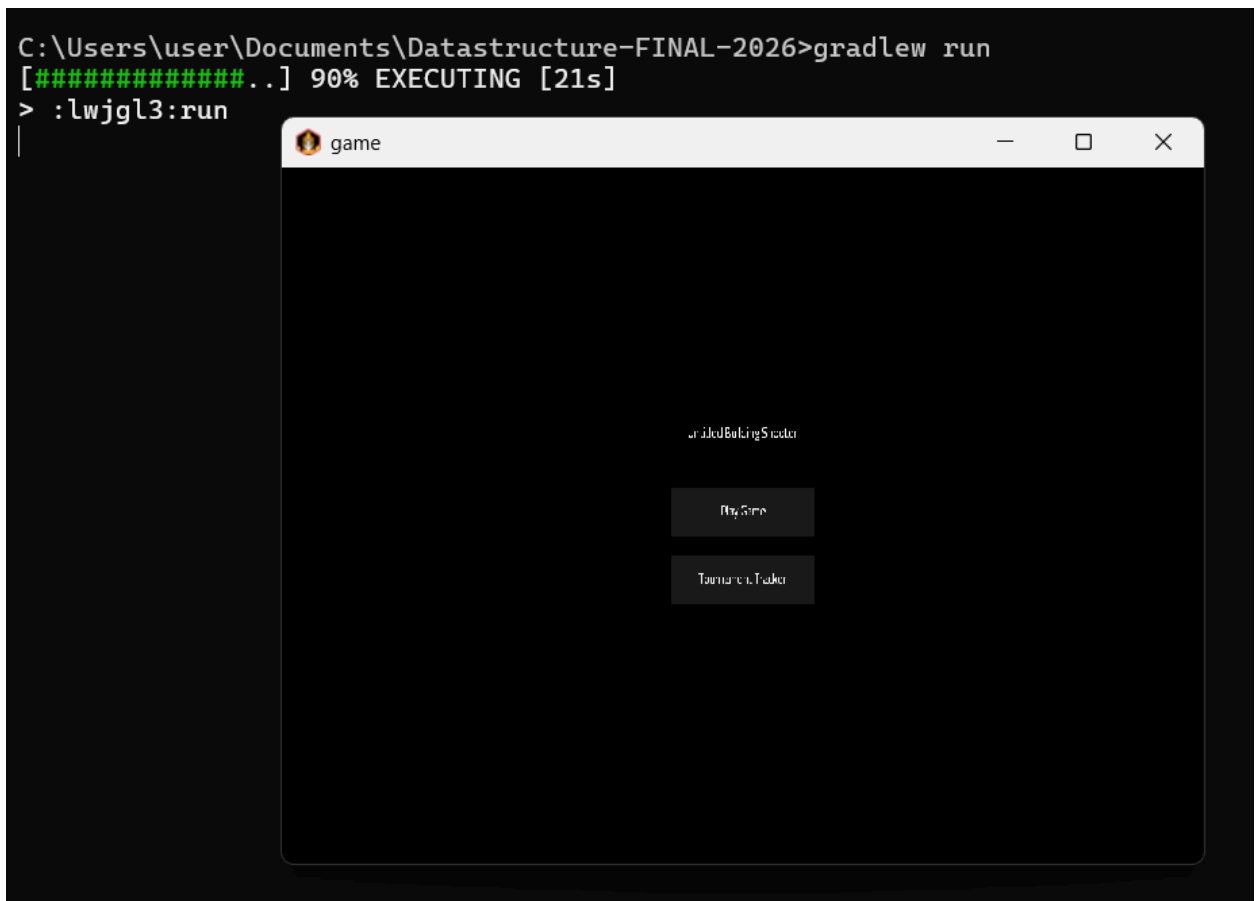
C:\Users\user\Documents>git clone https://github.com/dmr-uni-projects-assignments/Datastructure-FINAL-2026
Cloning into 'Datastructure-FINAL-2026'...
remote: Enumerating objects: 763, done.
remote: Counting objects: 100% (75/75), done.
remote: Compressing objects: 100% (43/43), done.
remote: Total 763 (delta 17), reused 56 (delta 12), pack-reused 688 (from 1)
Receiving objects: 100% (763/763), 432.80 KiB | 5.85 MiB/s, done.
Resolving deltas: 100% (181/181), done.

C:\Users\user\Documents>cd Datastructure-FINAL-2026

C:\Users\user\Documents\Datastructure-FINAL-2026>

```

3. Start a command prompt in the project directory
4. Run this following command:
gradlew run



The screenshot shows a terminal window with the following text:

```

C:\Users\user\Documents\Datastructure-FINAL-2026>gradlew run
[#####...] 90% EXECUTING [21s]
> :lwjgl3:run
|

```

Overlaid on the terminal is a game window titled "game". The game interface is dark-themed and displays the text "unlabeledBakingStudio" at the top. Below this text are two buttons: "Play Game" and "Tutorial & Tasker".

Appendix - Links

GitHub Repo: <https://github.com/dmr-uni-projects-assignments/Datastructure-FINAL-2026>

Presentation:

https://docs.google.com/presentation/d/e/2PACX-1vTERx2w9r-6Cq1c4FHRUM_j9OGv3cXM_pUo-nnwKD9HYhGaNESKCuarMEK3UB4fW00OTEQd00MT-Z6j/pub?start=false&loop=false&delayms=60000

